Expressing Parallelism with SYCL: Basic Data-Parallel Kernels

André Cerqueira

July 22, 2024

Este trabalho foi financiado por: Projeto 10110190 – EUROCC2, com apoio financeiro da FCT/MCTES através de fundos nacionais (PIDDAC).

Overview

- Understand work-items, work-groups, and ND-ranges.

Work-Items

- **Work-Item:** The smallest unit of work, similar to a thread in traditional parallel programming.
- Executes a single instance of the kernel function.
- Identified by a unique ID within the ND-range.

Work-Groups

- **Work-Group:** A collection of work-items.
- Work-items within a work-group can share data and synchronize with each other.
- Identified by a unique group ID within the ND-range.

ND-Range

- **ND-Range:** Defines the total number of work-items and how they are divided into work-groups.
- Specifies the dimensions and size of the computational grid.
- **Example:** An ND-range of `range<2>(16, 16)` with work-group size `range<2>(4, 4)` creates a 2D grid of 16x16 work-items divided into 4x4 work-groups.

ND-Range

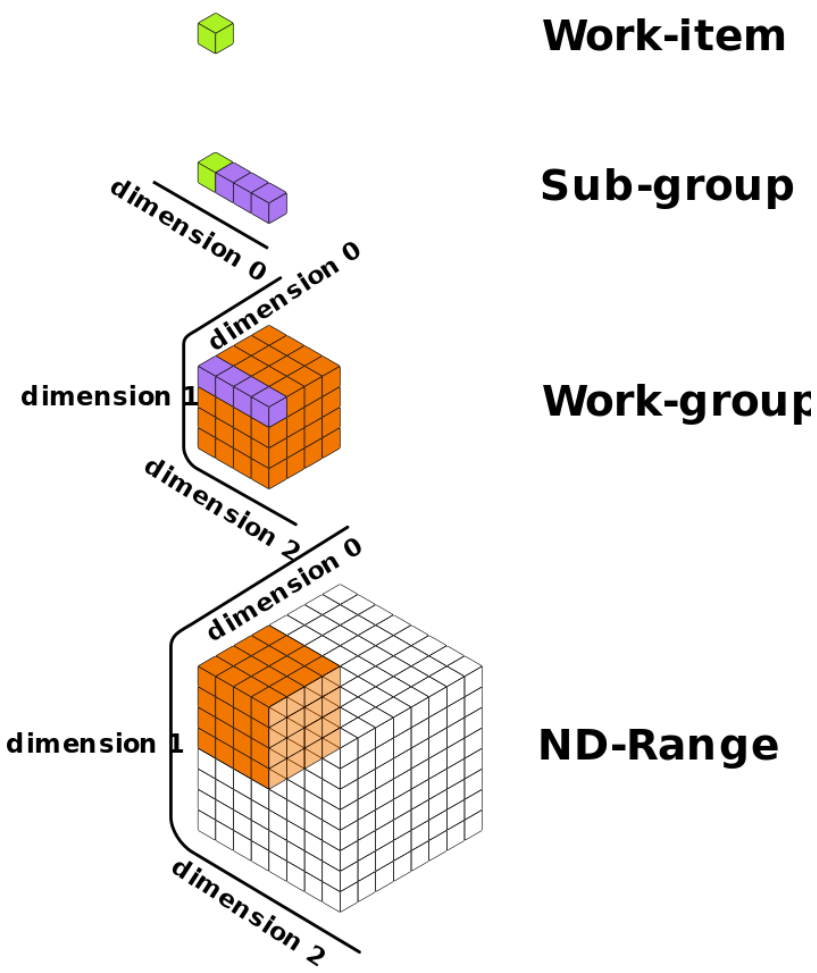

Figure: Illustrative image of ND-range

Basic Data-Parallel Kernels

- **Simple ‘parallel_for‘:** Execution over a 1-, 2-, or 3-dimensional range object.
- **ND-range ‘parallel_for‘:** Allows grouping instances of the kernel together.
- **Hierarchical Kernels:** Nesting kernel invocations.

Basic Data-Parallel Kernel Example

```
Q.submit([&](handler &cgh) {
    accessor acc { buf, cgh, write_only };

    cgh.parallel_for(range<2> { n_work_items }, [=](id<2> idx) {
        acc[idx] = 42.0;
    });
});

```

ND-range Data-Parallel Kernels

- ND-ranges are constructed using two ‘range‘ objects: global and local execution ranges.
- **Global range:** Total size of the ND-range.
- **Local range:** Size of each work-group.

Work-Groups and Work-Items in ND-ranges

- Work-groups have local memory accessible by all work-items within the group.
- Group-level barriers and fences allow synchronization within a work-group.
- Work-items in a work-group are scheduled concurrently to a single compute unit.

Matrix Multiplication

- Compute $C_{ij} = \sum_k A_{ik} B_{kj}$ using data-parallel kernel.
- Use a 2D range to define the execution space.
- Each work-item computes one element of the resulting matrix.

Matrix Multiplication Kernel

```
Q.submit([&](handler &cgh) {
    accessor A { bufA, cgh, read_only };
    accessor B { bufB, cgh, read_only };
    accessor C { bufC, cgh, write_only, no_init };

    cgh.parallel_for(range<2> {N, N}, [=](id<2> idx) {
        int row = idx[0];
        int col = idx[1];
        float sum = 0.0f;
        for (int k = 0; k < N; k++) {
            sum += A[row][k] * B[k][col];
        }
        C[row][col] = sum;
    });
});
```

- **Range:** Defines a 2D range for matrix dimensions.
- **Kernel:** Each work-item computes the dot product for one element.

Optimized Matrix Multiplication with ND-range (Code)

```
Q.submit([&](handler &cgh) {
    accessor A { bufA, cgh, read_only };
    accessor B { bufB, cgh, read_only };
    accessor C { bufC, cgh, write_only, no_init };

    range<2> global{N, N};
    range<2> local{B, B};

    cgh.parallel_for(nd_range<2>(global, local),
[=](nd_item<2> it) {
        int row = it.get_global_id(0);
        int col = it.get_global_id(1);
        float sum = 0.0f;
        for (int k = 0; k < N; ++k) {
            sum += A[row][k] * B[k][col];
        }
        C[row][col] = sum;
    });
});

```

Optimized Matrix Multiplication with ND-range (Explanation)

- **ND-range:** Improved locality of memory accesses.
- **Local memory:** Work-items within a group can share data.
- **Global range:** Total computation grid size.
- **Local range:** Size of each work-group.
- **nd_item:** Provides access to work-item and group information.

Summary

- Work-items are the basic units of execution.
- Work-groups are collections of work-items.
- ND-ranges define the computational grid structure.
- Data parallelism is achieved with ‘parallel_for‘ and kernel-based programming.